UNIVERSIDAD ARGENTINA DE LA EMPRESA

FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

**Materia:** Programación 1

**Profesor:** Abel Israel Laime Huanca

**Grupo:** 4

**Integrantes:**


- Citoler, Martín Alejo
- De Amorrortu, Belén
- Mottura Moreno, Diana Natalia
- Oven, Santiago Agustín
- Ruffo, Eduardo Gabriel


**Fecha:** 13-10-2025

**Ejercicio Grupal:** Reserva de turnos para clases de tenis

# 1. Índice


| | | |
|---|---|---|
| 1. | Índice | 1 |
| 2. | Ideas del proyecto | 2 |
| 3. | Roles | 2 |
| 4. | Objetivos | 3 |
| 5. | Menús de la aplicación | 3,4 |
| 6. | Estructura del proyecto | 5 |
| 7. | Ejemplos | 5 |
| 8. | Capturas de pantalla | 6,7 |
| 8. | Explicación general del código | 8,9,10 |
| 9. | Link al repositorio | 10 |
| 10. | Bibliografía o fuentes de consultas utilizada | 10 |

## 2. Idea del proyecto

Crear una aplicación por consola para gestionar la reserva de turnos de clases de tenis, incluyendo:

- Datos de los profesores
- Datos de los estudiantes de tenis
- Fechas disponibles
- Fechas ocupadas
- Alta, baja y modificación de la gestión de turnos para las clases.

## 3. Roles

- Project leader: Amorrortu Belén
- Desarrollador: Mottura Diana
- QA tester: Oven Santiago
- Analista funcional: Citoler Martìn
- Documentación: Ruffo Eduardo Gabriel

Tareas Realizadas:

- Gabriel Ruffo: Definición de la estructura del proyecto, archivos de almacenamiento necesarios y su contenido base. Desarrollo de funciones reutilizables para la lectura y escritura de archivos.
- Belén De Amorrortu: desarrollo de algoritmos de búsqueda, modificación y solicitud de datos para reutilizar en los distintos módulos de la aplicación.
- Santiago Oven: lectura, creación y modificación de Profesores mediante los métodos previamente definidos.
- Diana Mottura: lectura, creación y modificación de clientes mediante los métodos previamente definidos.
- Martín Citoler: lectura, creación y modificación de Turnos mediante los métodos previamente definidos.

## 4. Objetivos

- Crear una aplicación de consola que sirva para gestionar los turnos de las clases de tenis en una institución.
- Si bien la gestión será por consola, es posible a futuro con unos pocos cambios usarla con una interfaz gráfica.
- Desarrollar una gestión de clientes (alta, baja y modificación) que se guardará localmente.
- Desarrollar una gestión de turnos dentro de un calendario acotado que debe permitir alta, baja y modificación.
- Desarrollar validaciones de la información ingresada de cada usuario.
- Desarrollar validaciones para los turnos y fechas.
- Desarrollar lectura, modificación y escritura de archivos json o txt que se usarán para guardar de forma local la información de la aplicación.

## 5. Menús de la aplicación:

- --- MENÚ PRINCIPAL ---
- 1. Gestionar clientes
- 2. Gestionar profesores
- 3. Gestionar turnos
- 0. Salir
- Seleccione una opción:

- --- MENÚ CLIENTES ---
- 1. Listar clientes
- 2. Agregar cliente
- 3. Modificar cliente
- 4. Eliminar cliente
- 0. Volver
- Seleccione una opción:

- --- MENÚ PROFESORES ---
- 1. Listar profesores
- 2. Agregar profesor
- 3. Modificar profesor
- 4. Eliminar profesor
- 0. Volver
- Seleccione una opción:

- --- MENÚ TURNOS ---
- 1. Listar turnos
- 2. Agregar turno
- 3. Cancelar turno
- 4. matriz_turnos
- 0. Volver
- Seleccione una opción:

## 6. Estructura del proyecto

/Programacion1-GestionTurnosTenis

- /Data
  - clientes.json
  - profesores.json
  - turnos.json
- clientes.py
- main.py
- menu.py
- profesores.py
- storage.py
- turnos.py
- utils.py
- validaciones.py
- readme.md

## 7. Ejemplos

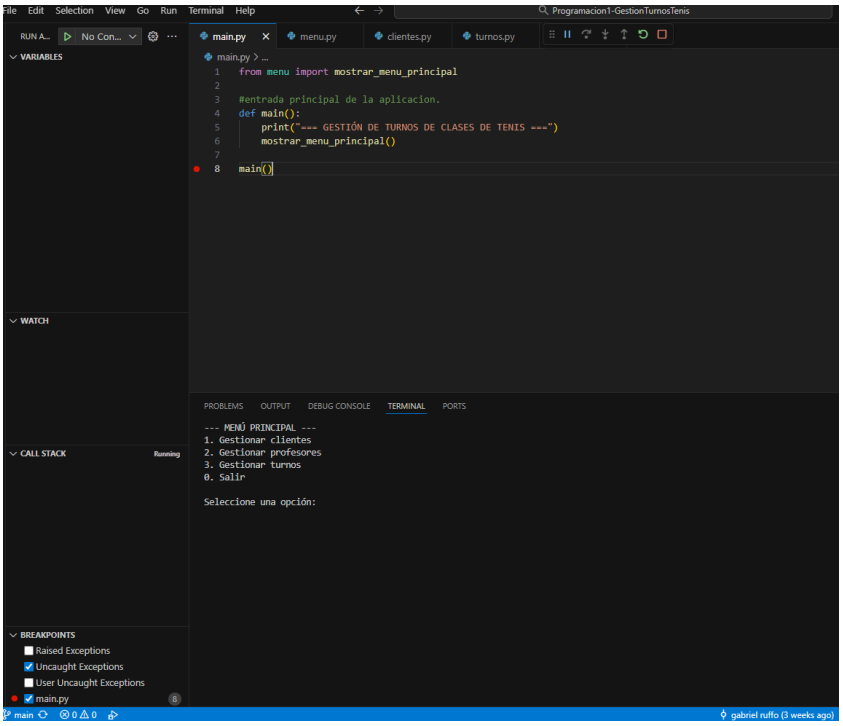
- Agregar Profesor
- Entrada: Nombre, Turno
- Salida: Profesor agregado al archivo Profesores.json y mensaje en consola.

- Eliminar Profesor
- Entrada: ID
- Salida: Profesor eliminado de Profesores.json con confirmación en consola.

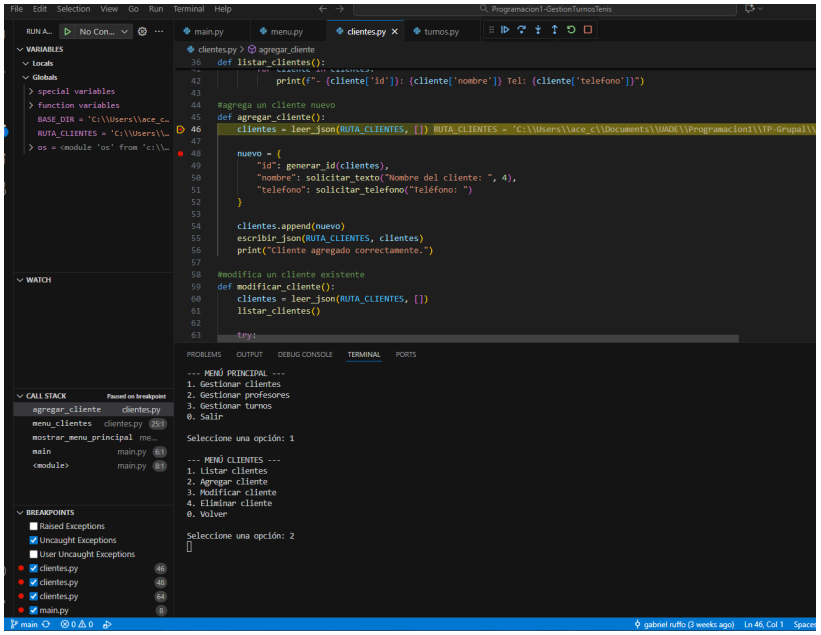
- Agregar Cliente
- Entrada: Nombre, Edad
- Salida: Cliente agregado a Clientes.json.

# 8. Capturas de pantallas.

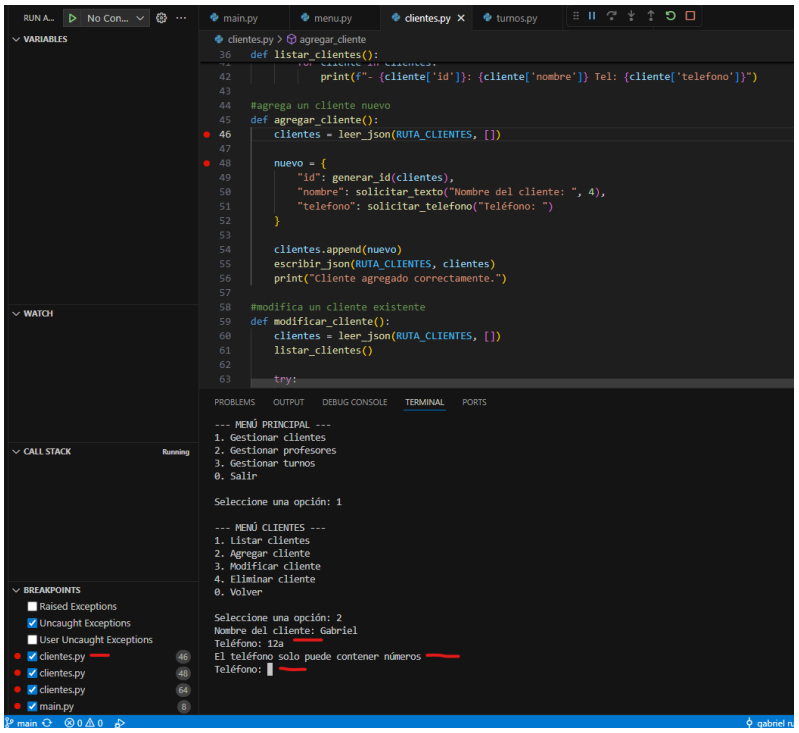
Menú principal



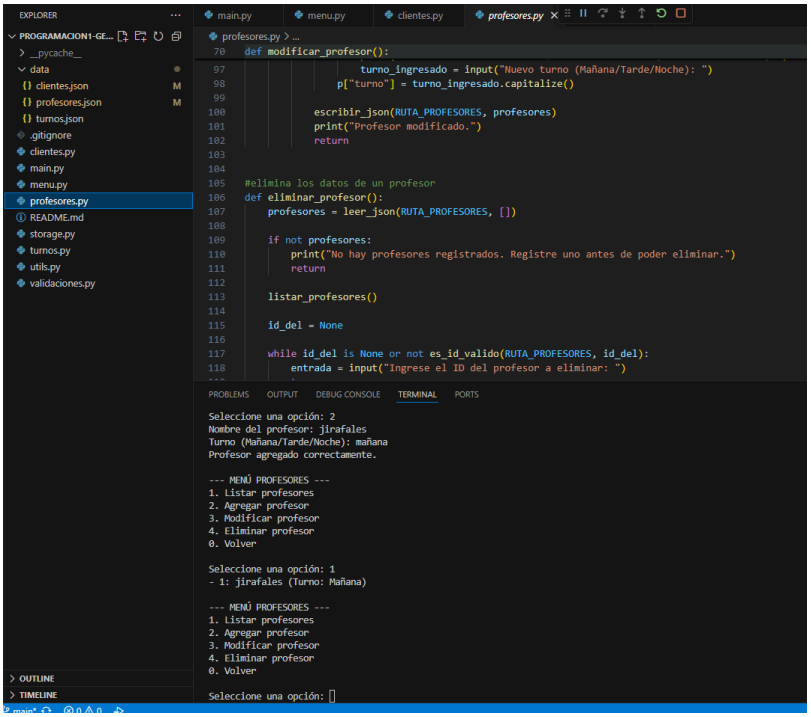

Ejemplo debug clientes.py




Ejemplo de pruebas para agregar numero de telefono y que no acepte letras




## Listado de profesores del codigo profesores.py



# 9. Explicación general del código.

**main.py**

- Inicio de la aplicación

**menu.py**

- Tiene la lógica para mostrar una lista de opciones para la gestión de turnos, clientes, profesores y la opción de salir de la aplicación.

**clientes.py**

- tiene el menú y la lógica para gestionar los clientes
  - **menu_clientes():** lista las opciones de gestión que hay para los clientes.
  - **listar_clientes()**: lista los clientes que hay guardados en clientes.json, si no hubiera clientes, se muestra el mensaje “No hay clientes registrados”
    - Para leer los clientes utiliza la función leer_json(ruta, []) que recibe 2 parámetros y devuelve una lista.
  - **agregar_cliente()**: lista los clientes existentes y agrega un nuevo cliente a ese listado, luego por medio de la función, escribir_json(ruta, []) guarda los datos, sobreescribiendo lo ya existente.
  - **modificar_cliente()**: Se trae toda la lista de clientes, Se muestra por consola, luego se pide el id del cliente para ser editado, se recorre toda la lista hasta encontrar el cliente, luego se pide los datos a ser modificados. Por último se reescribe el archivo clientes.json para actualizar los datos.
  - **eliminar_cliente():** Se lee el archivo de clientes.json. Se listan todos los clientes. Se pasa por consola el id a ser eliminado. Se recorre la lista de clientes, eliminando el cliente. Luego se guarda nuevamente la lista en el archivo clientes.json

**profesores.py**

- tiene la misma logica que clientes.py con algunas validaciones extra de disponibilidad del profesor

**turnos.py**

- **menu_turnos()**: muestra un listado de las opciones de gestión de turnos que hay.
- **listar_turnos()**: muestra un listado de turnos leídos en turnos.json por medio de la función leer_json()
- **agregar_turnos()**: lee los turnos existentes, lee los clientes existentes, lee los profesores existentes. A partir de esta información pide el ID del profesor, el id del cliente y la fecha del turno. Todo esto se valida con utils.py y si todo está bien, se guarda el nuevo turno en turnos.json por medio del método escribir_json()
- **cancelar_turno()**: lee los turnos existentes, los lista por consola. Se solicita el id del turno que se quiere dar de baja, y se actualiza la lista. Luego se la guarda en turnos.json
- **Matriz_turnos()**: crea la matriz a modo de tabla para visualizar los turnos junto con su respectivo cliente y profesor.

**utils.py**

- Contiene validaciones y métodos comunes a la gestión de datos leídos, solicitados y guardados.
- solicitar_entero(): solicita números y valida que sea numérico.
- solicitar_texto(): solicita un texto y lo valida (solo letras)
- solicitar_telefono(): solicita un teléfono y lo valida
- solicitar_fecha_hora(): solicita una fecha y la valida que sea correcta
- solicitar_id(): solicita un id
- agregar_registro(): agrega un registro en la lista
- eliminar_registro(): elimina un registro de la lista
- generar_id(): genera un id que no exista

**validaciones.py**

- trabaja en conjunto con utils.py hace las validaciones correspondientes de cada campo de entrada que tiene utils.py
- Los siguientes métodos validan tipo de dato y lógica de la información permitida, según corresponda.
    - es_texto_valido()
    - es_fecha_hora_valida()
    - es_telefono_valido():
    - es_id_existente():
    - validar_turnos():
    - es_id_valido():
    - es_nombre_valido()

## 10. Link al repositorio git.

- https://github.com/Gabriel-Ruffo-uade/Programacion1-GestionTurnosTenis

## 11. Bibliografía o fuentes de consultas utilizadas.


- UVAS vista en clase.
- Google, stackoverflow, chatGPT